

Templates : Template/PG-fr.docx | Template/Presentation_BELACEL_V3.pptx

PDF : TP_Informatique_ENS_1ère_année_AN_08.pdf

Instructor: BELACEL Madani

Module: Computer Science

Level: ENS — Year 1 — English Department

Duration: 1h30

Academic Year: 2025-2026

Learning Objectives

At the end of this lab, the student is able to:

- Design a complete Python mini-project from A to Z
- Structure a project into multiple files
- Implement an interactive user interface (menu)
- Apply best practices: functions, docstrings, error handling
- Produce a reusable pedagogical project for the classroom
- Import and export data in CSV format

Lab Plan

Phase	Duration	Activity
Project presentation	10 min	Architecture and structure
Exercise 1	15 min	Create the CSV question base
Exercise 2	15 min	CSV loading module
Exercise 3	20 min	Interactive menu interface
Exercise 4	15 min	Random selection and scoring
Exercise 5	15 min	Exporting results

Mini-Project Presentation

Topic: Create an Interactive Pedagogical Quiz on the computer science concepts covered in the module.

Expected features:

1. Load questions from a CSV file (4 categories, 15+ questions)
2. Ask questions in random order
3. Handle multiple categories (Architecture, Algorithms, Python, Networks)
4. Time the quiz
5. Calculate the score and display results

6. Export results to a CSV file

7. Command-line interface with menu

```

flowchart TD
    subgraph Project[Quiz Project Structure]
    A[quiz.py] --> B[questions.csv]
    A --> C[resultats.csv]
    end
    A --> D[Interactive menu]
    D --> E[Start quiz]
    D --> F[Show rules]
    D --> G[Quit]
    E --> H[Load questions]
    H --> I[Filter by category]
    I --> J[Shuffle randomly]
    J --> K[Ask questions]
    K --> L[Calculate score]
    L --> M[Display results]
    M --> N[Export to CSV]

```

Full Grading Scale

Part	Points	Details
CSV question base	4 pts	4 categories, 15+ questions, correct format
Loading module	4 pts	Load, filter, shuffle functions
Menu interface	4 pts	Clear menu, smooth navigation, error handling
Random selection and scoring	4 pts	Random order, score, grade, timer
Result export	4 pts	Clean CSV export, timestamp
Total	20 pts	

Exercise 1 — Create the CSV Question Base (15 min)

Problem statement: Create a file questions.csv with at least 15 questions across 4 categories (Architecture, Algorithms, Python, Networks). Format: Category, Question, Answer.

Solution:

```

import csv

questions = [
    ["Architecture", "What does CPU stand for?", "Central Processing Unit"],
    ["Architecture", "Is RAM volatile?", "Yes"],
    ["Architecture", "What model is the basis of modern computers?", "von Neumann"],
    ["Algorithms", "Which structure follows the LIFO principle?", "Stack"],
    ["Algorithms", "Which structure follows the FIFO principle?", "Queue"],
    ["Algorithms", "What is the complexity of bubble sort?", "O(n²)"],
    ["Python", "Which function displays text in Python?", "print"],
    ["Python", "Which keyword defines a function?", "def"],
    ["Python", "What type does input() return?", "str"],
    ["Python", "How to handle errors in Python?", "try/except"],
    ["Networks", "What does IP stand for?", "Internet Protocol"],
    ["Networks", "Which protocol secures HTTP?", "HTTPS"],
    ["Networks", "What does DNS stand for?", "Domain Name System"],
    ["Networks", "Which port does HTTP use?", "80"],

```

```
[
    ["Networks", "What does LAN stand for?", "Local Area Network"],
]

with open("questions.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["Category", "Question", "Answer"])
    writer.writerows(questions)

print(f"{len(questions)} questions created in questions.csv")
```

Exercise 2 — CSV Loading Module (15 min)

Problem statement: Create the question loading functions in `quiz.py`:

1. `load_questions(filename)` — loads the CSV and returns a list of dictionaries
2. `filter_by_category(questions, category)` — filters questions by category
3. `shuffle_questions(questions)` — randomly shuffles

Solution:

```
import csv
import random
import time
from datetime import date

def load_questions(filename):
    """Loads questions from a CSV file."""
    questions = []
    try:
        with open(filename, "r", encoding="utf-8") as f:
            reader = csv.DictReader(f)
            for row in reader:
                questions.append({
                    "category": row["Category"],
                    "question": row["Question"],
                    "answer": row["Answer"]
                })
    except FileNotFoundError:
        print(f"Error: file {filename} not found.")
        return []
    return questions

def filter_by_category(questions, category):
    """Filters questions by category."""
    if category == "All":
        return questions
    return [q for q in questions if q["category"] == category]

def shuffle_questions(questions):
    """Randomly shuffles questions."""
    random.shuffle(questions)
    return questions
```

Exercise 3 — Interactive Menu Interface (20 min)

Problem statement: Create the main quiz menu with the following options:

1. Start the quiz (select category)

2. Show rules

3. Quit

The menu must be clear, robust, and handle input errors.

```

flowchart TD
    A[Welcome screen] --> B[Main menu]
    B --> C[1. Start quiz]
    B --> D[2. Rules]
    B --> E[3. Quit]
    C --> F[Choose category]
    F --> G[Enter name]
    G --> H[Start quiz]
    H --> I[Show results]
    I --> J[Export?]
    J -->|Yes| K[Export CSV]
    J -->|No| B
    K --> B
    D --> L[Show rules]
    L --> B
    E --> M[End]

```

Solution:

```

def main_menu():
    """Displays the main menu and handles choices."""
    questions = load_questions("questions.csv")
    if not questions:
        return
    categories = ["All", "Architecture", "Algorithms", "Python", "Networks"]
    while True:
        print("\n" + "="*40)
        print("1. Start quiz")
        print("2. Rules")
        print("3. Quit")
        choice = input("Your choice: ")
        if choice == "1":
            for i, cat in enumerate(categories, 1):
                print(f"{i}. {cat}")
            try:
                c = int(input("Category (1-5): "))
                cat = categories[c - 1] if 1 <= c <= 5 else "All"
            except (ValueError, IndexError):
                cat = "All"
            qf = filter_by_category(questions, cat)
            print(f"Questions: {len(qf)} in '{cat}'")
            run_quiz(qf, input("Your name: "))
        elif choice == "2":
            print("• Text answer • 4 categories • Score /20 • CSV export")
        elif choice == "3":
            print("Goodbye!")
            break
        else:
            print("Invalid choice.")

```

Exercise 4 — Random Selection and Scoring (15 min)

Problem statement: Add quiz functions with random selection, scoring, and grade. The program shuffles questions, asks each question, checks the answer, and assigns a grade.

Solution:

```

def ask_question(question):
    """Asks a question and returns True if correct."""
    print(f"\n[{question['category']}] {question['question']}")
    answer = input("Your answer: ").strip().lower()
    expected = question['answer'].strip().lower()
    if answer == expected:
        print("✓ Correct!")
        return True
    else:
        print(f"x Wrong. Answer: {question['answer']}")
        return False

def run_quiz(questions, student_name=""):
    """Runs the quiz with random shuffle and timer."""
    if not questions:
        print("No questions to ask.")
        return

    questions = shuffle_questions(questions)
    score = 0
    total = len(questions)
    start = time.time()

    print(f"\n=== Quiz started! ({total} questions) ===")
    for i, q in enumerate(questions, 1):
        print(f"\nQuestion {i}/{total}")
        if ask_question(q):
            score += 1

    elapsed = time.time() - start
    show_results(score, total, elapsed)
    export_choice = input("\nExport results? (y/n): ")
    if export_choice == "y" and student_name:
        export_results(score, total, elapsed, student_name)

def show_results(score, total, elapsed):
    """Displays results with grade."""
    percentage = (score / total * 100) if total > 0 else 0
    print("\n" + "="*40)
    print(" RESULTS")
    print("="*40)
    grade = round(score / total * 20, 2) if total > 0 else 0
    print(f"Score: {score}/{total} ({percentage:.1f}%)")
    print(f"Grade: {grade}/20")
    print(f"Time: {elapsed:.1f}s")

    if percentage == 100:
        remark = "Perfect!"
    elif percentage >= 80:
        remark = "Very good!"
    elif percentage >= 60:
        remark = "Good!"
    elif percentage >= 40:
        remark = "Could do better."
    else:
        remark = "Needs review."
    print(f"Remark: {remark}")

```

Exercise 5 — Exporting Results (15 min)

Problem statement: Add the export function that saves results (date, name, score, total, percentage, time) to `resultats.csv` with automatic timestamp.

```

flowchart LR
    A[Quiz finished] --> B{Export?}
    B -->|Yes| C[Open resultats.csv]
    B -->|No| D[Return to menu]
    C --> E{File empty?}
    E -->|Yes| F[Write header]
    E -->|No| G[Skip]
    F --> H[Add result]
    G --> H
    H --> I[✓ Exported!]
    I --> D

```

Solution:

```

def export_results(score, total, elapsed, student_name, filename="resultats.csv"):
    """Exports quiz results to a CSV file."""
    with open(filename, "a", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        if f.tell() == 0:
            writer.writerow(["Date", "Student", "Score", "Total",
                             "Percentage", "Grade_20", "Time_s"])
        grade = round(score / total * 20, 2) if total > 0 else 0
        percentage = f"{score/total*100:.1f}" if total > 0 else "0"
        writer.writerow([date.today(), student_name, score, total,
                         percentage, grade, f"{elapsed:.1f}"])
        print(f"Results exported to {filename}")

def main():
    """Program entry point."""
    show_welcome()
    try:
        with open("questions.csv", "r", encoding="utf-8"):
            pass
    except FileNotFoundError:
        print("\n⚠ questions.csv file not found. Run exercise 1 first.")
        return
    main_menu()

def show_welcome():
    """Displays the welcome screen."""
    print("="*40)
    print(" Interactive Pedagogical Quiz")
    print("="*40)

if __name__ == "__main__":
    main()

```

Self-Assessment MCQ

- Which function randomly shuffles a list? a) `random.shuffle()` ✓ b) `random.choice()` c) `random.sample()`
- What is the correct format for a CSV file? a) Comma-separated data ✓ b) Space-separated data c) JSON format data
- What does `csv.DictReader()` do? a) Reads each line as a dictionary ✓ b) Writes dictionaries c) Converts a list to a dictionary
- In the mini-project grading scale, how many points is the question base worth? a) 2 points b) 4 points ✓ c) 6 points
- Which Python module gets today's date? a) `datetime` ✓ b) `time` c) `calendar`

Mini-Project Grading Rubric

Criterion	Weight	Not acquired (0)	Partial (0.5)	Acquired (1)
Complete functionality	20%	Program does not run	Runs partially	Runs perfectly
Structured code	15%	No functions	Some functions	Well-organized functions
Error handling	15%	No handling	Partial handling	try/except + validation
CSV files	15%	No CSV	Read or write only	Correct read and write
User interface	15%	No menu	Basic menu	Clear and functional menu
Documentation	10%	None	Minimal comments	Docstrings + comments
Project originality	10%	Exact copy	Light adaptations	Personalized project

Session Schedule — 1h30 (90 min)

Phase	Duration	Cumulative total
Project presentation	10 min	10 min
Exercise 1 — Question base	15 min	25 min
Exercise 2 — Loading module	15 min	40 min
Exercise 3 — Menu interface	20 min	60 min
Exercise 4 — Selection and scoring	15 min	75 min
Exercise 5 — Export and finalization	15 min	90 min
Total	90 min	

Pedagogical Note ENS

Teaching transposition

This mini-project is the culmination of the module. It mobilizes all the skills acquired: variables, conditions, loops, functions, lists, dictionaries, CSV files, and error handling.

- For future teachers: This quiz is a directly reusable tool. They can adapt the questions to their subject (history, languages, sciences) and use the CSV export to track student progress.
- In secondary school: Simplify by keeping only a single category of questions.

Trainer advice

Guide students in structuring the project: splitting into functions, separate data file, error handling. The project can be done in pairs (pair programming). Evaluate code quality (comments, naming) as much as functionality. For high school students, lighten the requirements by removing the CSV export.

Extension

Transform the quiz into a web application with Flask (ENS Year 2 curriculum). Add an SQLite database to store questions and user scores. This capstone project can become the subject of the ENS1 project presented in session 10.